

## SIMATS ENGINEERING ASSESSMENT TOOL 2

COURSE CODE: CSA14

COURSE NAME: Compiler Design

---

### COURSE OUTCOME COVERED

**CO1: Apply lexical analysis for token specification and recognition using LEX tool (BL3)**

*Assessment Tool Weightage: 20%*

---

QUESTIONS: 3

Total Marks:50

Annexure A

### ERROR ANALYSIS TASK

---

#### *Code of Conduct:*

I Deepak Rao. L <sup>192421170</sup> (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.  
I understand that any violation of academic integrity rules will result in disciplinary action.

*Signature of the student:*

**ERROR ANALYSIS TASK**

Q1. The following program is intended to perform simple arithmetic operations, function calls, array access, and looping constructs. However, the code contains **multiple errors introduced at different phases of compilation, including lexical, syntax, semantic, logical, and runtime errors.**

Given Program

```
#include <stdio.h>
```

```
int sumArray(int arr[], int n) {  
    int i, sum = 0;  
    for(i = 0; i <= n; i++) {  
        sum = sum + arr[i];  
    }  
    return sum;  
}
```

```
int main() {  
    int arr[5] = {1,2,3,4,5}  
    int total;  
  
    total = sumArray(arr);  
  
    if(total == 15) {  
        printf("Correct Sum\n")  
    }  
}
```

```
int a = 10, b = 0;  
int c = a / b;
```

```
int *p;  
*p = 20;
```

```
return 0;  
}
```

Tasks:

(a) Error Identification (Analysis Level)

1. Identify all the errors present in the program. For each error:
2. Specify the line number
3. Clearly describe the nature of the error

(b) Classification of Errors (Understanding + Analysis)



1. Classify each identified error into the appropriate compiler phase:

2. Lexical Errors
3. Syntax Errors
4. Semantic Errors
5. Logical Errors
6. Runtime Errors

(c) Compilation Behavior (Evaluation Level)

1. Analyze which errors will be detected during compilation and which will only appear during execution.
2. Explain how early-stage errors (lexical/syntax) may prevent detection of later-stage errors.

(d) Code Correction (Application Level)

1. Rewrite the corrected version of the program such that:
2. All errors are removed
3. The program executes correctly
4. Proper syntax and semantics are maintained

(e) Impact Analysis (Evaluation Level)

1. Discuss the impact of at least three errors on program execution.
2. Explain how incorrect constructs like improper pointer usage, division by zero, and infinite loops affect system behavior.

(f) Advanced Compiler Perspective (Create Level)

1. Suggest how a compiler can be improved to:
2. Provide better error messages
3. Perform early detection of runtime errors
4. Propose at least two enhancements in compiler design for efficient debugging.

Q2. The following C program contains **multiple lexical errors** introduced intentionally. These errors occur due to **invalid tokens, incorrect identifiers, malformed constants, and illegal symbols**, which are typically detected during the **lexical analysis phase** of a compiler. Carefully examine the program and answer the questions given below.

```
#include <stdio.h>

int main() {
    int 1num = 100;
    float rate@ = 25.5;
    char ch = 'abc';
    int total = 50$20;
    int num#1 = 30;
    int %count = 40;

    printf("Welcome to Compiler Design);

    int x = 09;
    int y = 0xZ12;
    int z = 12.3.4;
```



int x = 10;  
int y = 11;

int z = 12;  
int w = 13;

int v = 14;  
int u = 15;

int t = 16;  
int s = 17;

printf("Value of x is: %d", x);

return 0;

1. Lexical

(a) Token Identification

- 1) Identify all lexical units in the program.
- 2) For the first iteration and the invalid token.

(b) Token Classification

- 1) For each token, explain why it is considered a lexical unit.
- 2) Specify which rule of token classification is violated.
- 3) Classification rules
- 4) Character classification
- 5) Whitespace/separator token rules
- 6) Operator/keyword rules

(c) Token Connection

- 1) Ensure each invalid token has a valid token.
- 2) Example: 2value → value2

(d) Compiler Behavior Analysis

- 1) Program has a lexical analyzer (scanner) processes each token.
- 2) If scanner detects the compiler.
- 3) Reports immediately
- 4) Continues scanning using error recovery

(e) Advanced Thinking

- 1) Design a simple rule (or LPA idea) to detect invalid identifiers starting with digits.
- 2) Suggest how error messages can be improved for beginners.

(f). The following C program is designed to perform arithmetic operations, function calls, conditional checks, pointer usage, and looping. However, the program contains multiple errors distributed across different phases of a compiler, including:

1. Lexical Errors
2. Syntax Errors
3. Semantic Errors
4. Intermediate Code Generation Issues (logic/precedence)
5. Code Optimization Issues (redundancy/inefficiency)
6. Runtime / Target Code Errors



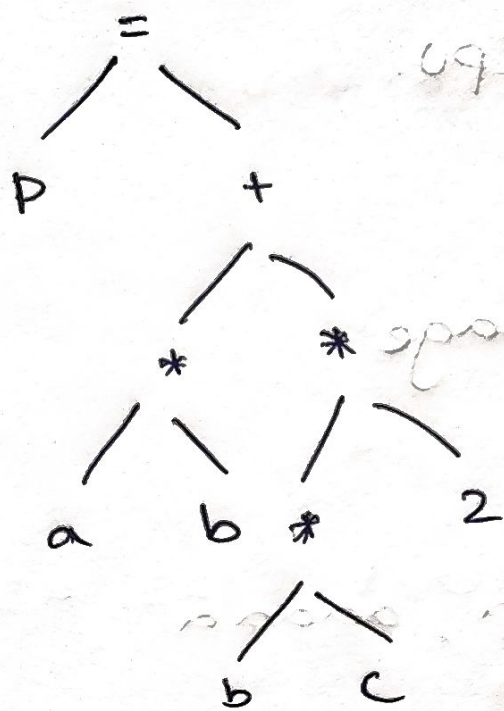
Trace the processing of the expression

a) Tokens

$P = (a * b) + (b * c) * 2$

Expression:  $P = (a * b) + (b * c) * 2$

b) Syntax tree



c) Intermediate code:

$t_1 = a * b$

$t_2 = b * c$

$t_3 = t_2 * 2$

$t_4 = t_1 + t_3$

$P = t_4$



ne and member func  
structor is a special me

\*) LR(0)

d) Optimized code:

$$t1 = a * b$$

$$t2 = b * c$$

$$p = t1 + (t2 * 2)$$

e) Target code:

The optimized intermediate code is converted into machine / assembly code, which is executed efficiently by the CPU.

Q2. Construct the Language

Example:

a) a, aa, aaa, aaaa, aaaaa

b) b, bb, bbb, bbbb, bbbbbb

c) ab, abab, abbb, abbbb, ababab

d) 0, 00, 000, 0000, 00000

e) 1, 11, 111, 1111, 11111

Q3. Lexical Analysis

1) Number of Tokens

given C program contains 22 tokens

## i) Lexical Errors

- \* 1x0ab is an invalid integer literal because identifiers cannot start with a digit
- \*  $a = \&x;$  cause a type mismatch, since  $a$  is a  $\text{char}^*$  and  $\&x$  is an  $\text{int}^*$ .

## Regular Expression

i) Even number of 0's:

$$1^*(01^*01^*)^*$$

ii) At least one 1:

$$(0|1)^*1(0|1)^*$$

iii) odd number of 0's number of 1's

$$(100)^*1(11)^*0)^* \quad (\text{Any equivalent RE is acceptable})$$

iv) starts with 1 and ends with 0:

$$1(0|1)^*0$$

v) No consecutive 1's:

$$(0|10)^*1?$$



## Q5. Lexical Analyzer

### a) Number of Tokens

The given input contains 20 tokens

### b) Token categories

\* Keywords: int, float, if, return

\* Identifiers: var123, var456

\* Operators: +, -

\* Separators: (, ), {, }, ;

\* Integer literal: 12345

\* Floating point literal: 123.45

### c) FSM state Transitions

The lexical analyzer changes state while record each token. Based on the input, the token number of state transitions is approximately 62. The transitions include recognizing keywords, identifiers, literals, operators, and separators.